CAIRN: Information-Theoretic, Leakage-Resilient Threshold Custody of Master Keys from Linear Algebra over $\mathbb{F}_{2^{127}-1}$

Ahmedouyahya

Mathematics for Systems Security, Cryptographer track

November 2026

Abstract

A master key is exposed through five different windows: *at rest* (a stolen backup), *over time* (an attacker who collects pieces patiently), *in memory* (cold boot, swap, partial memory disclosure), *during reconstruction* (timing and cache side channels) and *under manipulation* (a custodian who lies). We present CAIRN, a t-of-n custody scheme that addresses all five with one algebraic toolbox and *no computational assumption at all*: no block cipher, no hash function, no discrete logarithm. The key is encoded into the Mersenne field $\mathbb{F}_p$, $p = 2^{127} - 1$, armored with an algebraic manipulation detection tag, shared with a Vandermonde system, and every share coordinate is stored as an inner product $\langle L, R \rangle$ of two vectors kept in separate memory regions. Shares are re-randomized by Gibbs steps built from random skew-symmetric matrices, and moved to fresh sharings by zero-polynomial refreshes. We prove perfect secrecy against $t - 1$ custodians, tamper detection with error at most $(k + 1)/p \approx 2^{-125}$, resilience to 378 bits of arbitrary split-state leakage from a share coordinate at advantage 2^{-128}, and security against mobile adversaries. We report our own cryptanalysis, including a cross-epoch attack on the natural two-step refresh that we found and fixed. Every claim is backed by an exact or statistical experiment that `make verify` reproduces: exhaustive enumeration in small fields (77,525 dealings, 13,432,176 secret/error pairs), byte-identical agreement with an independent reference implementation, a fixed-versus-random timing test (CAIRN $|t| \leq 1.6$, textbook baseline $|t|$ up to 11,848.1), an assembly audit and memory-safety checks. The implementation is a `no_std`, `unsafe`-free Rust core with a hardened command line tool.

1 Introduction

Whoever holds a master key holds everything it protects: a signing key for software releases, the root of a password manager, the key-encryption key of a database. The classical answer to "who should hold it" is threshold secret sharing (Shamir [16], Blakley [2]): split the key into n shares so that any t recover it and any $t - 1$ reveal nothing. Shamir's scheme is perfect, but only *as mathematics*. Deployed, it leaves five windows open, and each has been exploited in practice or in the literature.

W1. At rest. Shares are files; files get copied.

W2. Over time. A *mobile* adversary [13] who steals one share a month eventually holds t shares of the same polynomial.

W3. In memory. Shares and the key sit in RAM, swap and core dumps, readable by cold-boot attacks [8] or partial memory disclosure.

W4. During reconstruction. Variable-time arithmetic leaks through timing and cache channels [10].

W5. Under manipulation. A single cheating custodian who submits a modified share makes everyone else recover a wrong key while she computes the right one (Tompa and Woll [17]).

Contributions. CAIRN is a complete custody scheme in which each window is closed by a layer with a proof and a measurement (Table 1).

- A construction that uses only linear algebra and polynomial arithmetic over $\mathbb{F}_p$, $p = 2^{127} - 1$: algebraic manipulation detection (AMD) [4], Vandermonde sharing, inner-product split-state storage [5, 6], and a new inversion-free refresh by random skew-symmetric matrices (Section 3).
- Unconditional security theorems with explicit bounds, proved in full (Section 4), including a self-contained proof of the split-state leakage bound (Theorem 7) and an exact analysis of cross-epoch leakage of the refresh (Theorem 10).

Window	Layer	Guarantee	Evidence
W1 at rest	L3 Vandermonde sharing	$t-1$ shares independent of the key (Theorem 2)	exhaustive, Table 3
W2 over time	L5 zero-sharing refresh	mobile adversary learns nothing (Theorem 3)	demo, Section 5.4
W3 in memory	L4 split-state storage	$\le 2^{-128}$ per coordinate under 378 bits of leakage (Theorem 7)	exact, Fig. 3
W3 over refreshes	L4 skew-symmetric refresh	cross-epoch advantage $p^{-(\ell-1)}$ (Theorem 10)	exact, Table 5
W4 reconstruction	L6 constant-time combine	no secret-dependent branch, address or latency	dudect, ctgrind, asm
W5 manipulation	L2 AMD armor	wrong key accepted w.p. $\le (k+1)/p$ (Theorem 5)	exhaustive, Table 4

Table 1: Vulnerability windows of a master key and how Cairn closes them.

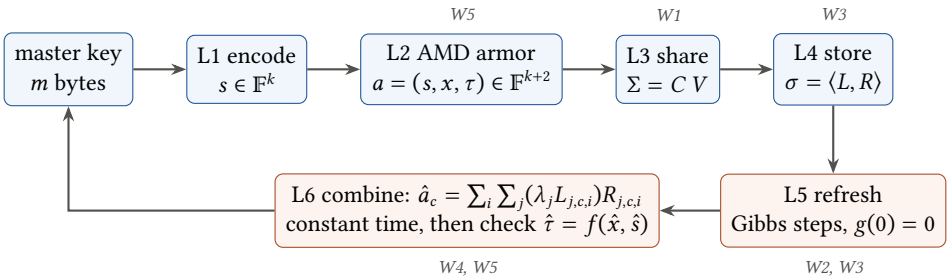


Figure 1: The six layers of Cairn and the windows (W1 to W5) they close.

- Cryptanalysis of our own design (Section 5): a complete break of the obvious two-step refresh, the limits of any keyless robust scheme, and attacks on parsing, timing and process memory.
- Evidence for every claim (Section 6): exhaustive verification at small parameters through the production code path, cross-implementation known-answer tests, dudect-style timing tests, ctgrind, an assembly audit and Miri.

2 Preliminaries

Field. Let $p = 2^{127}-1$, a Mersenne prime, and $\mathbb{F} = \mathbb{F}_p$. Since $2^{127} \equiv 1 \pmod p$, a 254-bit product $h \cdot 2^{128} + l$ reduces as $l \bmod 2^{127} + \lfloor l/2^{127} \rfloor + 2h$ followed by one conditional subtraction, which we implement with a mask. Inversion is $x^{-1} = x^{p-2}$ (Fermat) with the *public* exponent $p-2$, so its operation sequence does not depend on x; we set $0^{-1} = 0$. All elements are kept in canonical form $[0, p)$, and encodings that are not canonical are rejected, so every element has exactly one byte representation.

Notation. $[n] = \{1, \dots, n\}$. For $u, v \in \mathbb{F}^\ell$, $\langle u, v\rangle = \sum_i u_i v_i$ and $u^\perp = \{v : \langle u, v\rangle = 0\}$. $\mathrm{Skew}_\ell(\mathbb{F}_p)$ is the set of $\ell \times \ell$ matrices with $A^\top = -A$. For distributions P, Q on a finite set, $\mathrm{TV}(P,Q) = \frac{1}{2}\sum_z |P(z) - Q(z)|$; the advantage of any distinguisher between P and Q is at most $\mathrm{TV}(P,Q)$. $X \xleftarrow{\$} S$ denotes uniform sampling. Characters of $(\mathbb{F}, +)$ are $\psi_a(z) = \omega^{az}$ with $\omega = e^{2\pi i/p}$.

Threat model. The adversary may (A1) steal up to $t-1$ share files; (A2) be *mobile*, stealing up to $t-1$ shares in each refresh epoch, where a custodian corrupted during a refresh counts for both epochs; (A3) control up to $t-1$ custodians and submit arbitrary share files for them at combination time; (A4) obtain adaptive leakage of at most λ_L bits from the L memory region and λ_R bits from the R region of any share (*split-state* model); (A5) obtain one storage half before a refresh and the other half after it; (A6) measure the timing and cache behaviour of the combiner; (A7) feed malformed files. We trust: (T1) the dealer during dealing, (T2) the combiner during combination (it outputs the key by definition), (T3) the operating system's random number generator, (T4) confidentiality of refresh deltas in transit, and (T5) the binding between a custodian and her share index at combination time (the roster; Theorem 11 shows that no keyless scheme can avoid an assumption of this kind).

3 The Cairn construction

L1, encoding. A secret of $m \le 65535$ bytes is prefixed with its 16-bit length, padded with zeros to a multiple of 15 bytes and cut into k big-endian 120-bit limbs $s_1, \dots, s_k < 2^{120} < p$. The map is injective, and decoding rejects any vector that is not exactly an encoding. A parameter $k_{\min}$ pads short secrets to a fixed number of limbs so that k, which is public, does not reveal the length. A 32-byte key gives $k = 3$.

Algorithm 1: IPEnc(σ): inner-product encoding

Input: $\sigma \in \mathbb{F}$, dimension $\ell \geq 2$

$L_1 \xleftarrow{\$} \mathbb{F} \setminus \{0\}$; $L_2, \ldots, L_\ell \xleftarrow{\$} \mathbb{F}$; $R_2, \ldots, R_\ell \xleftarrow{\$} \mathbb{F}$;

$R_1 \leftarrow (\sigma - \sum_{i \geq 2} L_i R_i) \cdot L_1^{p-2}$; ▷ inverts a random value, constant time

return (L, R); ▷ uniform on $S_\sigma = \{(L, R) : \langle L, R \rangle = \sigma,\ L_1 \neq 0\}$

Algorithm 2: Refresh(L, R): three Gibbs half-steps

$A, B, C \xleftarrow{\$} \mathrm{Skew}_\ell(\mathbb{F}_p)$; ▷ $\ell(\ell-1)/2$ field elements each

$L \leftarrow L + AR$; ▷ $\langle AR, R \rangle = 0$, so $\langle L, R \rangle$ is unchanged

$R \leftarrow R + BL$;

$L \leftarrow L + CR$;

return (L, R)

L2, AMD armor [4]. Draw $x \xleftarrow{\$} \mathbb{F}$ and set

$$\tau = f(x, s) = x^{k+2} + \sum_{i=1}^{k} s_i x^i, \qquad a = (s_1, \ldots, s_k, x, \tau) \in \mathbb{F}^D, \quad D = k + 2.$$

The tag is evaluated by Horner's rule as $x \cdot (s_1 + x(s_2 + \cdots + x(s_k + x \cdot x)))$: exactly $k + 1$ multiplications whatever the values.

L3, sharing. For each component $c \in [D]$ draw $f_c(X) = a_c + \sum_{m=1}^{t-1} r_{c,m} X^m$ with $r_{c,m} \xleftarrow{\$} \mathbb{F}$. In matrix form, with $C \in \mathbb{F}^{D \times t}$ the coefficient matrix and $V = [\, j^m \,]_{0 \leq m < t,\ j \in [n]}$ a Vandermonde matrix, the share matrix is $\Sigma = CV$; share j is the column $\sigma_j = (f_c(j))_c$.

L4, split-state storage. Each coordinate $\sigma_{j,c}$ is stored as a pair $(L, R) \in \mathbb{F}^\ell \times \mathbb{F}^\ell$ with $\langle L, R \rangle = \sigma_{j,c}$, produced by Algorithm 1. All L vectors of a share live in one memory region and file section, all R vectors in another.

L5, refresh. Algorithm 2 re-randomizes a pair without computing its value; Algorithm 3, in the spirit of proactive secret sharing [9], moves all custodians to a fresh sharing of the same key.

L6, combination. Algorithm 4 interpolates at zero in constant time and checks the AMD tag. Lagrange coefficients depend only on the public indices.

4 Security analysis

Theorem 1 (Correctness)**.** *For every secret and every t-subset T of honest shares of the same epoch, Algorithm 4 returns the secret.*

Proof. Every operation of Algorithms 1 and 2 preserves $\langle L, R \rangle$: Algorithm 1 by construction, and each line of Algorithm 2 adds a vector orthogonal to the other half because $\langle Mw, w \rangle = w^\top M^\top w = -w^\top M w$ forces $\langle Mw, w \rangle = 0$ for skew-symmetric M when p is odd. In Algorithm 3, $\langle L, R + (\delta / L_1) e_1 \rangle = \langle L, R \rangle + \delta$, and $\sum_{j \in T} \lambda_j g_c(j) = g_c(0) = 0$, so refreshes do not change the interpolated value. Hence $\hat{a}_c = \sum_{j \in T} \lambda_j f_c(j) = f_c(0) = a_c$ by Lagrange interpolation, the tag verifies, and Decode inverts L1. □

4.1 Secrecy at rest and over time

Theorem 2 (Perfect secrecy)**.** *For every set $B \subseteq [n]$ with $|B| \leq t - 1$, the joint distribution of the share files $(\mathit{share}_j)_{j \in B}$ is the same for every secret.*

Proof. Fix a component c. The values on B are $f_c(j) = a_c + \sum_{m=1}^{t-1} r_{c,m} j^m$, that is $a_c \mathbf{1} + V_B r_c$ with $V_B = [j^m]_{j \in B,\, 1 \leq m < t}$. Now $V_B = \mathrm{diag}(j)_{j \in B}\, W$, where $W = [j^{m-1}]$ is a $|B| \times (t-1)$ Vandermonde matrix on distinct points, hence of full row rank, and $\mathrm{diag}(j)$ is invertible because $0 \notin B$. So $r_c \mapsto V_B r_c$ is onto $\mathbb{F}^{|B|}$ and maps the uniform distribution to the uniform distribution; the values are uniform whatever a_c (the Reed-Solomon view of Shamir sharing [12]). Components use independent coefficients, and the files are randomized functions of the values, the public parameters and the random identifier. □

Algorithm 3: ProactiveRefresh for a dealing with parameters (t, n, D)

Coordinator: for $c \in [D]$ draw $g_c(X) = \sum_{m=1}^{t-1} u_{c,m} X^m$, $u_{c,m} \xleftarrow{\$} \mathbb{F}$; send $\delta_j = (g_c(j))_c$ to custodian j confidentially;
Custodian j**:** for $c \in [D]$: $R_{j,c,1} \leftarrow R_{j,c,1} + \delta_{j,c} \cdot L_{j,c,1}^{-1}$; $(L_{j,c}, R_{j,c}) \leftarrow \mathsf{Refresh}(L_{j,c}, R_{j,c})$;
increment the epoch; erase δ_j;

Algorithm 4: Combine(shares $\mathcal{S}$)

check that all shares have the same dealing identifier, parameters and epoch, distinct indices in $[n]$, and $|\mathcal{S}| \geq t$;
foreach *t-subset $T \subseteq \mathcal{S}$ in lexicographic order (at most 2^{16})* **do**
  $\lambda_j \leftarrow \prod_{m \in T \setminus \{j\}} m\,(m-j)^{-1}$ for $j \in T$; ▷ `public`
  for $c \in [D]$ **do**
    $\hat{a}_c \leftarrow \sum_{i=1}^{\ell} \sum_{j \in T} (\lambda_j L_{j,c,i})\, R_{j,c,i}$; ▷ `no single share is ever formed`
  if ct_eq$(\hat{\tau}, f(\hat{x}, \hat{s}))$ (declassified verdict) **then**
    report as inconsistent every $u \notin T$ for which $(T \setminus \{\min T\}) \cup \{u\}$ fails the check;
    return Decode$(\hat{s})$
return $\perp$

Theorem 3 (Mobile adversary)**.** *Let B_e be the custodians corrupted in epoch e, with $|B_e| \leq t-1$, where a custodian corrupted during the refresh from $e-1$ to e belongs to both B_{e-1} and B_e and also sees her delta. Assume deltas are confidential and erased. Then the adversary's view over any number of epochs is independent of the secret.*

Proof. Write $f_c^{(e)}$ for the epoch-e polynomial, so $f_c^{(e)} = f_c^{(e-1)} + g_c^{(e-1)}$ with $g_c^{(e-1)}(0) = 0$ fresh. We show by induction that the epoch-e part of the view is independent of the secret given the earlier view. Epoch 0 is Theorem 2. For $e \geq 1$, the vector $(g_c^{(e-1)}(j))_{j \in B_e}$ is uniform on $\mathbb{F}^{|B_e|}$ (same Vandermonde argument, as $|B_e| \leq t-1$) and independent of everything before. The new values are: for $j \in B_e \cap B_{e-1}$, old value plus seen delta, a function of the earlier view and of uniform fresh values; for $j \in B_e \setminus B_{e-1}$, $f_c^{(e-1)}(j) + g_c^{(e-1)}(j)$, which is uniform and independent of the earlier view because the second term is a fresh uniform coordinate that appears nowhere else in the view. Storage encodings are fresh randomized functions of these values. □

4.2 Tamper detection

Lemma 4 (AMD bound [4])**.** *Let $k + 2 \not\equiv 0 \pmod{p}$. For every $s \in \mathbb{F}^k$ and every non-zero $\Delta = (d, e, g) \in \mathbb{F}^k \times \mathbb{F} \times \mathbb{F}$, $\Pr_{x \xleftarrow{s} \mathbb{F}} \left[\, \tau + g = f(x+e, s+d) \,\right] \leq \frac{k+1}{p}$, where $\tau = f(x, s)$.*

Proof. The event is $P(x) = 0$ for $P(x) = f(x+e, s+d) - f(x, s) - g$. If $e \neq 0$, the coefficient of x^{k+1} in $(x+e)^{k+2} - x^{k+2}$ is $(k+2)e \neq 0$ and the other terms have degree at most k, so P has degree exactly $k+1$ and at most $k+1$ roots. If $e = 0$, $P(x) = \sum_i d_i x^i - g$ has degree at most k and is the zero polynomial only if $d = 0$ and $g = 0$, i.e. $\Delta = 0$. □

Theorem 5 (Robustness)**.** *Let at most $t-1$ custodians be corrupted (A3), each submitting an arbitrary file under her own index (T5). Then Algorithm 4 returns either the correct secret or $\perp$, except with probability at most $N\,(k+1)/p$, where $N \leq 2^{16} + n$ is the number of subsets it evaluates.*

Proof. For one subset T the reconstruction is linear: $\hat{a} = a + \sum_{j \in T \cap B} \lambda_j (\sigma'_j - \sigma_j) = a + \Delta_T$. The corrupted parties choose σ'_j from their view, which by Theorem 2 is independent of a; in particular, for any fixed s, Δ_T is independent of x. If $\Delta_T = 0$ the output is correct. Otherwise, by Theorem 4, the tag verifies with probability at most $(k+1)/p$. A union bound over the evaluated subsets concludes. Since every t-subset contains an honest share when $|B| \leq t-1$, the adversary cannot replace the reconstruction by a value of her choice. □

With $k = 3$ the bound is 2^{-125} per subset and below 2^{-108} for the whole search. The same argument shows that shares from different epochs combine to the correct secret or to $\perp$ except with probability $(k+1)/p$: the mixture adds $\sum_j \lambda_j (g(j) - g'(j))$, a function of refresh randomness only.

4.3 Leakage from storage

Lemma 6 (Inner-product extractor)**.** *Let $\mathcal{A}, \mathcal{B} \subseteq \mathbb{F}^\ell$ be non-empty and $X \xleftarrow{\$} \mathcal{A}$, $Y \xleftarrow{\$} \mathcal{B}$ independent. Then $\mathrm{TV}\big(\langle X, Y \rangle,\, U_{\mathbb{F}}\big) \leq \frac{1}{2}\sqrt{p^{\ell+1}/(|\mathcal{A}||\mathcal{B}|)}$.*

Proof. For $a \neq 0$, by Cauchy-Schwarz and then extending the sum to all of $\mathbb{F}^\ell$,

$$\Big|\sum_{x\in\mathcal{A}}\sum_{y\in\mathcal{B}}\psi_a(\langle x,y\rangle)\Big|^2 \le |\mathcal{A}|\sum_{x\in\mathbb{F}^\ell}\Big|\sum_{y\in\mathcal{B}}\psi_a(\langle x,y\rangle)\Big|^2 = |\mathcal{A}|\sum_{y,y'\in\mathcal{B}}\sum_{x\in\mathbb{F}^\ell}\psi_a(\langle x,y-y'\rangle) = |\mathcal{A}||\mathcal{B}|p^\ell,$$

so $|\mathbb{E}\,\psi_a(\langle X,Y\rangle)| \le \sqrt{p^\ell/(|\mathcal{A}||\mathcal{B}|)}$ (Lindsey's lemma [3]). Writing $D = P_{\langle X,Y\rangle} - U_{\mathbb{F}}$, Parseval gives $\sum_z D(z)^2 = \frac{1}{p}\sum_{a\neq 0}|\mathbb{E}\,\psi_a(\langle X,Y\rangle)|^2$, and Cauchy-Schwarz gives $\mathrm{TV} = \frac{1}{2}\sum_z |D(z)| \le \frac{1}{2}\sqrt{p\sum_z D(z)^2} \le \frac{1}{2}\sqrt{(p-1)p^\ell/(|\mathcal{A}||\mathcal{B}|)}$. □

Theorem 7 (Split-state leakage). *Let a leakage protocol adaptively obtain at most λ_L bits about L and λ_R bits about R, each query being a function of one half and of earlier answers. For all $\sigma,\sigma' \in \mathbb{F}$, the transcripts $\Lambda_\sigma, \Lambda_{\sigma'}$ on $\mathsf{IPEnc}(\sigma)$ and $\mathsf{IPEnc}(\sigma')$ satisfy*

$$\mathrm{TV}(\Lambda_\sigma,\Lambda_{\sigma'}) \le \left(1+\tfrac{1}{p}\right)2^{(\lambda_L+\lambda_R)/2}\,p^{-(\ell-3)/2}.$$

Proof. Consider the *independent world*: $X \xleftarrow{\$} \mathcal{A}_0 = \{x : x_1 \neq 0\}$, $Y \xleftarrow{\$} \mathbb{F}^\ell$, $Z = \langle X,Y\rangle$. Then Z is exactly uniform (as $X \neq 0$), and conditioned on $Z=\sigma$ the pair (X,Y) is uniform on S_σ, which is exactly the output distribution of $\mathsf{IPEnc}(\sigma)$. Each transcript value θ of the protocol corresponds to a rectangle $\mathcal{A}_\theta \times \mathcal{B}_\theta$ (conditioning a product distribution on events of the form $\{h(X)=\cdot\}$ or $\{h(Y)=\cdot\}$ keeps it a uniform product), there are at most 2^λ transcripts with $\lambda = \lambda_L+\lambda_R$, and the rectangles partition $\mathcal{A}_0\times\mathbb{F}^\ell$. By Theorem 6 and Cauchy-Schwarz,

$$\varepsilon := \mathrm{TV}\big((\Lambda,Z),(\Lambda,U)\big) \le \sum_\theta \frac{|\mathcal{A}_\theta||\mathcal{B}_\theta|}{|\mathcal{A}_0|p^\ell}\cdot\frac{1}{2}\sqrt{\frac{p^{\ell+1}}{|\mathcal{A}_\theta||\mathcal{B}_\theta|}} \le \frac{\sqrt{p^{\ell+1}}\sqrt{2^\lambda|\mathcal{A}_0|p^\ell}}{2|\mathcal{A}_0|p^\ell} = \frac{1}{2}\,2^{\lambda/2}\sqrt{\frac{p^{2-\ell}}{p-1}}.$$

Since Z is uniform, $\mathrm{TV}((\Lambda,Z),(\Lambda,U)) = \frac{1}{p}\sum_\sigma \mathrm{TV}(\Lambda_\sigma,\Lambda)$, so $\mathrm{TV}(\Lambda_\sigma,\Lambda) \le p\,\varepsilon$ for every σ, and the triangle inequality gives $\mathrm{TV}(\Lambda_\sigma,\Lambda_{\sigma'}) \le 2p\varepsilon = 2^{\lambda/2}p^{(3-\ell)/2}\sqrt{p/(p-1)}$. □

Corollary 8. *With $p = 2^{127}-1$ and $\ell = 8$, any $\lambda_L+\lambda_R \le 378$ bits of leakage give advantage at most 2^{-128} for one coordinate. A full share with D coordinates, whose L halves share one memory region and R halves another, tolerates $\lambda_L+\lambda_R \le 378$ bits leaked from the two regions in total at advantage $D\cdot 2^{-128}$: a hybrid over coordinates, in which the reduction samples the other coordinates itself and hard-wires them into the leakage functions, so each step faces the same total budget. Leakage from one half alone is harmless without limit: L is sampled independently of σ, and R alone is within $p^{-(\ell-1)}$ of independent.*

4.4 Leakage across refreshes

Lemma 9 (Gibbs step). *For $w \neq 0$ and $M \xleftarrow{\$} \mathrm{Skew}_\ell(\mathbb{F}_p)$, the vector Mw is uniform on $w^\perp$.*

Proof. $M \mapsto Mw$ is linear with image in $w^\perp$. It is onto: for $v \in w^\perp$ pick u with $\langle u,w\rangle = 1$ and let $M = vu^\top - uv^\top$; then $Mw = v\langle u,w\rangle - u\langle v,w\rangle = v$. A surjective linear map sends the uniform distribution to the uniform distribution. □

Hence each line of Algorithm 2 replaces one half by a uniform element of the affine hyperplane $\{L : \langle L,R\rangle = \sigma\}$ (or $\{R : \langle L,R\rangle = \sigma\}$), i.e. it is an exact Gibbs-sampler step for the uniform distribution on pairs with inner product σ. In particular the stored pair stays distributed as a fresh encoding (up to the $L_1 \neq 0$ condition, which changes it by at most $1/p$), so Theorem 7 applies in every epoch.

Lemma 10 (Cross-epoch leakage). *Let $(L,R) \leftarrow \mathsf{IPEnc}(\sigma)$ and $(L'',R') \leftarrow \mathsf{Refresh}(L,R)$. (i) If $\mathsf{Refresh}$ stops after its first two lines, $\langle L',R\rangle = \sigma$: the old R and the new L reveal σ. (ii) For the three-line refresh and all $\sigma,\sigma' \neq 0$, $\mathrm{TV}\big((R,L'')_\sigma,(R,L'')_{\sigma'}\big) = \mathrm{TV}\big((L,R')_\sigma,(L,R')_{\sigma'}\big) = p^{-(\ell-1)}$.*

Proof. (i) $L' = L + AR$ and $\langle AR,R\rangle = 0$. (ii) The marginal law of R is the same for all $\sigma \neq 0$: the number of L with $L_1 \neq 0$ and $\langle L,R\rangle = \sigma$ is $p^{\ell-1}-p^{\ell-2}$ if $(R_2,\dots,R_\ell) \neq 0$ and $p^{\ell-1}$ otherwise, independently of σ. Given R, by Theorem 9 L' is uniform on $H_R = \{u : \langle u,R\rangle = \sigma\}$, then R' uniform on $\{v : \langle L',v\rangle = \sigma\}$, then L'' uniform on $\{u : \langle u,R'\rangle = \sigma\}$. Counting the paths to a fixed $z \neq 0$: an L' not parallel to z admits $p^{\ell-2}$ vectors R' with $\langle L',R'\rangle = \langle z,R'\rangle = \sigma$; $L' = z$ admits $p^{\ell-1}$; $L' = cz$ with $c \neq 1$ admits none. At most one $L' \in H_R$ is parallel to z, and it equals z exactly when $\langle z,R\rangle = \sigma$. Therefore

$$\Pr[L'' = z \mid R] = p^{-3(\ell-1)}\Big[\big(p^{\ell-1} - [\langle z,R\rangle \neq 0]\big)p^{\ell-2} + [\langle z,R\rangle = \sigma]\,p^{\ell-1}\Big],$$

in which σ appears only in the last indicator. Summing $|\Pr_\sigma - \Pr_{\sigma'}|$ over the $2p^{\ell-1}$ vectors z with $\langle z,R\rangle \in \{\sigma,\sigma'\}$ gives $\mathrm{TV} = p^{-(\ell-1)}$ for every R. The pair (L,R') is three Gibbs steps apart in the same way ($R \mid L, L' \mid R, R' \mid L'$), and the same computation applies. □

Attack	Outcome against Cairn	Where
Exhaustive search with $t-1$ shares	every key equally likely; nothing to search	Theorem 2, Table 3
Tompa-Woll cheating [17]	detected ($\leq 2^{-125}$), cheater identified with $t+1$ shares	Theorem 5, Section 5.1
Mobile adversary [13]	mixed epochs refused, forged epochs caught by AMD	Theorem 3, Section 5.4
Old R + new L (two-step refresh)	**complete break of the naive design**; fixed by a third half-step	Theorem 10
Partial memory disclosure	$D \cdot 2^{-128}$ up to 378 bits per share; one half alone useless	Theorem 7, Fig. 3
Timing / cache	no leakage detected ($\lvert t\rvert \leq 1.6$ over $2 \cdot 10^6$ traces)	Fig. 4
Index forgery, fake dealing	impossible to prevent without keys; roster required	Theorem 11
Malleable or ambiguous files	canonical encoding, strict parser, never panics	property tests
Swap, core dumps, `ptrace`	`mlock`, `PR_SET_DUMPABLE=0`, `RLIMIT_CORE=0`	Section 6
Weak randomness	OS CSPRNG only; failure aborts (fail closed)	Section 6
Length leakage	optional padding to a fixed number of limbs	L1

Table 2: Attacks considered and their outcome.

When $\sigma = 0$ is one of the two values, the exact computation of Table 5 gives at most $(p+1)p^{-(\ell-1)}$ in every configuration we enumerated. At production size all of these are below 2^{-761}. Theorem 10 covers one refresh between the two observations; extending the exact analysis to adversaries that alternate halves over many epochs is left open, and we point to the continuous-leakage results of Dziembowski and Faust [6] for a related refresh.

4.5 Constant-time execution

Every operation on secret data in Algorithms 1, 2 and 4 is a fixed sequence of 64-bit multiplications, additions, shifts and masks: no division, no table lookup, no branch and no memory index depends on a secret. Conditional subtraction uses $\text{mask} = 0 - (t \gg 127)$ and a select $b \oplus (\text{mask} \wedge (a \oplus b))$, with values passed through an optimization barrier so that the compiler cannot turn the mask back into a jump. Loop bounds depend only on (t, n, ℓ, D); Lagrange coefficients only on public indices. The loop order of Algorithm 4 (coordinate outside, share inside) means that no partial sum ever equals a single share value. Exactly three secret-derived predicates are declassified, each by design: the AMD verdict, the event $L_1 = 0$ during a refresh (probability 2^{-127}), and the length of the decoded secret, which the caller sees anyway.

5 Cryptanalysis

We attacked our own design under the threat model of Section 2. Table 2 lists the attacks and their outcomes; the most instructive are discussed below.

5.1 Tompa-Woll cheating

Against plain Shamir with $t = 3$, a custodian who submits $\sigma_1 + \Delta$ makes the group recover $s' = s + \lambda_1\Delta$; knowing λ_1 and Δ, she computes $s = s' - \lambda_1\Delta$ and is the only one who holds the key. Our script `attacks/tompa_woll.py` carries this out against plain Shamir, then against the real `cairn` binary: the same edit produces an integrity failure and zero bytes of output, with a fourth share the key is recovered and share 1 is named, and 200 random tamperings of another share were never accepted. A property test applies random byte edits to one or two of three share files 3000 times and checks that a wrong secret is never returned.

5.2 The two-step refresh break

The natural refresh $L \leftarrow L + AR$, $R \leftarrow R + BL$ preserves $\langle L, R\rangle$ and looks complete. It is not: the new L is still orthogonal-shifted with respect to the *old* R, so an attacker who dumps the R region on Monday and the L region on Tuesday computes $\langle L_{\text{new}}, R_{\text{old}}\rangle = \sigma$ exactly (Theorem 10(i)). Our lab found this by exact enumeration through the production code: total variation 1 in every configuration (Fig. 2). A third half-step reduces the advantage to exactly $p^{-(\ell-1)}$, matching the closed form of Theorem 10(ii) to every printed digit.

5.3 What no keyless scheme can do

Proposition 11. *In any threshold scheme in which the combiner holds no secret, an adversary who may submit shares under arbitrary indices can make the combiner output any value s^* of her choice.*

p	t	n	dealings	sets		viol.	fail
				$t-1$	t		
13	2	4	169	4	6	0	0
13	3	5	2,197	10	10	0	0
13	4	6	28,561	20	15	0	0
31	3	4	29,791	6	4	0	0
7	5	6	16,807	15	6	0	0

Table 3: Perfect secrecy and recovery, every dealing enumerated (Theorem 2).

p	k	(s, Δ) pairs	worst	bound	tight
31	1	923,490	2/31	2/31	yes
13	1	28,548	2/13	2/13	yes
13	2	4,826,640	3/13	3/13	yes
7	2	117,600	3/7	3/7	yes
7	3	5,764,458	4/7	4/7	yes
11	2	1,771,440	3/11	3/11	yes

Table 4: AMD forgery probability, every secret and error enumerated (Theorem 4).

		2-step	3-step, $\sigma, \sigma' \neq 0$		predicted	3-step, all σ	
p	ℓ	(old R, new L)	(old R, new L)	(old L, new R)	$p^{-(\ell-1)}$	measured	$(p+1)p^{-(\ell-1)}$
3	3	1.0000	0.1111	0.1111	0.1111	0.3855	0.4444
5	2	1.0000	0.2000	0.2000	0.2000	0.8720	1.2000
5	3	1.0000	0.0400	0.0400	0.0400	0.2259	0.2400
3	4	1.0000	0.0370	0.0370	0.0370	0.1414	0.1481

Table 5: Exact cross-epoch total variation distance of the refresh (Theorem 10). Rows with $\ell = 2$ and $\ell = 3$ at $p \leq 5$ were also computed by driving the production code through every skew-symmetric matrix; both methods agree.

Proof. All dealing parameters are public, so she runs the dealing algorithm on s^* and submits t of the resulting shares. □

Robustness therefore needs the combiner to know which custodian holds which index (assumption T5). `cairn split` writes a public roster for this purpose. Under T5, Theorem 5 holds; without it, forged indices can also turn the additive error into an affine one, outside the scope of Theorem 4, which is why we state T5 explicitly instead of hoping it away.

5.4 Mobile adversary

`attacks/mobile_adversary.py` steals share 1 in January, lets the custodians refresh, and steals shares 2 and 3 in March. `cairn` refuses the mixture (epoch mismatch). With the epoch header forged, the mixture fails the AMD check, as predicted after Theorem 5. The honest custodians still recover the key.

6 Implementation and evaluation

Code. The core (`cairn-core`) is `no_std` Rust with `#![forbid(unsafe_code)]`, about 1500 lines. Every algorithm is generic over a `Field` trait, so the lab runs *the production code* in toy fields. Secret buffers are zeroized on drop and field elements never implement `Debug` output. The command line tool `cairn` adds the only `unsafe` code (six libc calls in one module): `PR_SET_DUMPABLE`= 0, `RLIMIT_CORE`= 0 and `mlock`/`mlockall`. It generates keys that never touch disk (`-generate`), writes shares with mode 0600 and never overwrites, refuses to print a key to a terminal, and hands a recovered key to another program through a pipe (`-exec`). The format is strict: fixed header, canonical elements, exact length.

Independent reference. `reference/cairn_ref.py` is a plain big-integer transcription of Algorithms 1 to 4. Driven by the same deterministic stream, it and the Rust code produce byte-identical share files for five parameter sets, before and after a proactive refresh (`kat/vectors.json`).

Exact verification. The lab enumerates every case at small parameters (Tables 3 to 5). Perfect secrecy holds with zero violations over 77,525 dealings; the AMD bound holds over 13,432,176 secret/error pairs and is attained, so Theorem 4 cannot be improved; the refresh analysis agrees with Theorem 10 exactly.

Timing. We follow dudect [15]: fixed and random inputs are interleaved in random order, inputs are prepared in advance in one array so both classes execute identical code, and Welch's t is computed on the raw cycle counts and on five percentile crops. On an Intel Core i7-1185G7, Cairn shows max $|t| = 1.6$ (multiplication, 2,000,000 traces), 1.5 (inversion) and 1.4 (full reconstruction and AMD check), all below the 4.5 threshold, while the textbook baseline

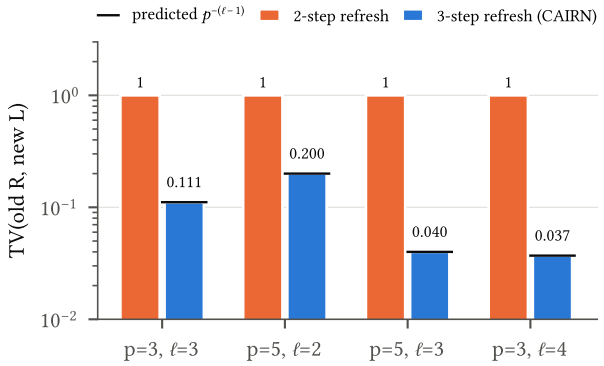


Figure 2: Cross-epoch leakage: two-step refresh (orange) leaks the value; Cairn's three-step refresh meets the prediction $p^{-(\ell-1)}$ (black ticks).

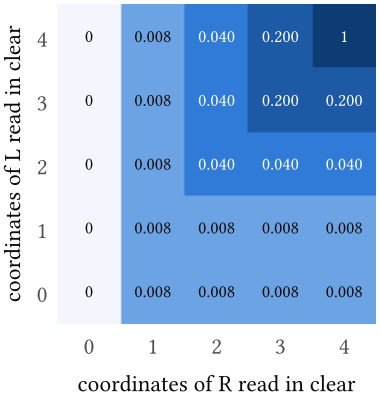


Figure 3: Exact advantage over $\mathbb{F}_5$, $\ell = 4$, when a coordinates of L and b of R are read in the clear. It is exactly 0 when R is not read ($b = 0$), equals $p^{-(\ell-\max(\min(a,b),1))}$ otherwise, and reaches 1 only when both halves are read in full.

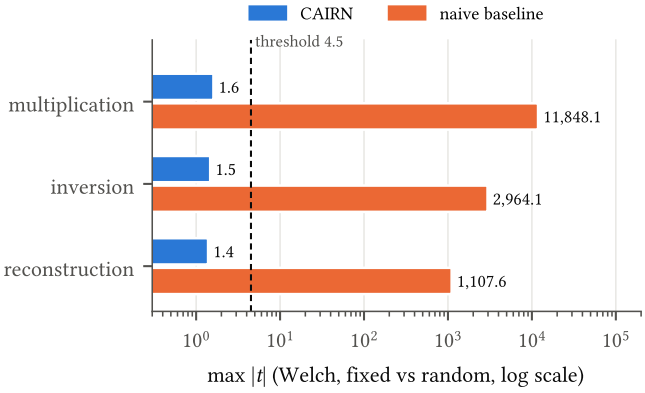


Figure 4: Fixed-versus-random Welch test (log scale). Bars below 4.5 show no detectable timing leakage.

operation (32-byte key)	median
field multiplication	25 ns
field inversion (Fermat, 127-bit exponent)	2.2 µs
split 32-byte key, 3-of-5, $\ell = 8$	63.6 µs
combine 3 shares (verify + decode)	9.3 µs
combine 4 shares, 1 cheater (identify)	45.9 µs
proactive refresh of all 5 shares	133.3 µs
local re-randomisation of one share	15.9 µs
generate deltas + apply one delta	28.2 µs

Table 6: Performance, one core, i7-1185G7.

(double-and-add multiplication, extended Euclid) shows 11,848.1, 2,964.1 and 1,107.6 on the same harness (Fig. 4). The statistic varies from run to run: over all campaigns we ran with this harness, the largest value for Cairn on any operation was about 2.1, well below 4.5, and the baseline never fell below 1,000. A methodological note: our first harness chose the input with a branch just before the timed region and reported a false positive ($|t| \approx 22$) on Cairn's multiplication; moving to dudect's precomputed-input design removed it. We report this because a timing test is only as good as its harness.

ctgrind, assembly and memory safety. Following Langley [11], `tools/ctgrind.sh` marks share memory as undefined for Valgrind's memcheck, which then reports any branch or address that depends on it. Over reconstruction, the AMD check, local refresh, inversion and encoding with poisoned inputs, memcheck reports 0 errors for Cairn; on the baseline (the positive control, which shows the poisoning is effective) it reports 2,289 errors in 3 code locations, all in the bit-by-bit branches of double-and-add multiplication. `tools/asm_audit.sh` disassembles the release binary: multiplication (72 instructions), addition and subtraction (110) and constant-time selection (29) contain no conditional jump and no division; the AMD tag contains exactly two, both on the public limb count. All core unit tests pass under Miri.

7 Comparison

Table 7 places Cairn among classical threshold schemes. Its distinguishing combination is unconditional tamper detection, a leakage model for storage and two levels of refresh, all without a computational assumption and without any pre-existing cipher or hash.

	Shamir [16]	Feldman VSS [7]	Pedersen VSS [14]	Asmuth-Bloom [1]	Cairn
Secrecy	perfect	computational (DL)	perfect	statistical	perfect
Tamper detection	none	yes, DL	yes, DL	none	yes, unconditional
Leakage model	none	none	none	none	split-state, bounded
Refresh	no	extensions	extensions	no	yes, two levels
Assumptions	none	discrete log	discrete log	none	none
Uses a cipher or hash	no	no	no	no	no

Table 7: Cairn against classical threshold schemes. VSS schemes additionally let custodians verify their shares against public commitments, which Cairn does not provide.

8 Limitations

We state what Cairn does *not* do. (1) The dealer and the combiner are trusted while they run; at the moment of use the key exists in the clear in the combiner's memory, in a locked, zeroized buffer, for about 9.3 µs. (2) Robustness requires authenticated indices (Theorem 11). (3) The leakage theorem bounds leakage from storage; it does not by itself model leakage of the computation that combines both halves, which we address with constant-time code and measurements instead. (4) We did not perform power or electromagnetic measurements on hardware. (5) The multi-epoch cross-refresh analysis beyond one refresh is open (Theorem 10). (6) Shares are not publicly verifiable as in VSS. (7) The `Copy` field type can leave register or stack copies that zeroization cannot reach; we minimize but cannot exclude them in a high-level language.

9 Conclusion

Linear algebra over one Mersenne prime is enough to protect a master key against theft, patience, memory disclosure, timing and lies, with theorems that need no unproven assumption and an implementation whose every claim is reproduced by one command. The most useful finding of our own cryptanalysis is also the simplest: a refresh that looks correct can leak everything across two snapshots, and only an exact analysis, not testing for correctness, reveals it.

Reproducibility. `make verify` runs the lint, unit, property and known-answer tests, the exhaustive lab, the attacks, ctgrind, the assembly audit, regenerates every figure and table in this paper from `results/*.json`, and rebuilds this PDF.

References

[1] C. Asmuth and J. Bloom. A modular approach to key safeguarding. *IEEE Trans. Inf. Theory*, 29(2):208–210, 1983.

[2] G. R. Blakley. Safeguarding cryptographic keys. In *AFIPS National Computer Conference*, pages 313–317, 1979.

[3] B. Chor and O. Goldreich. Unbiased bits from sources of weak randomness and probabilistic communication complexity. *SIAM J. Comput.*, 17(2):230–261, 1988.

[4] R. Cramer, Y. Dodis, S. Fehr, C. Padró and D. Wichs. Detection of algebraic manipulation with applications to robust secret sharing and fuzzy extractors. In *EUROCRYPT 2008*, LNCS 4965, pages 471–488.

[5] F. Davì, S. Dziembowski and D. Venturi. Leakage-resilient storage. In *SCN 2010*, LNCS 6280, pages 121–137.

[6] S. Dziembowski and S. Faust. Leakage-resilient cryptography from the inner-product extractor. In *ASIACRYPT 2011*, LNCS 7073, pages 702–721.

[7] P. Feldman. A practical scheme for non-interactive verifiable secret sharing. In *FOCS 1987*, pages 427–437.

[8] J. A. Halderman et al. Lest we remember: cold boot attacks on encryption keys. In *USENIX Security 2008*, pages 45–60.

[9] A. Herzberg, S. Jarecki, H. Krawczyk and M. Yung. Proactive secret sharing, or: how to cope with perpetual leakage. In *CRYPTO 1995*, LNCS 963, pages 339–352.

[10] P. Kocher. Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems. In *CRYPTO 1996*, LNCS 1109, pages 104–113.

[11] A. Langley. ctgrind: checking that functions are constant time with Valgrind. https://github.com/agl/ctgrind, 2010.

[12] R. J. McEliece and D. V. Sarwate. On sharing secrets and Reed-Solomon codes. *Commun. ACM*, 24(9):583–584, 1981.

[13] R. Ostrovsky and M. Yung. How to withstand mobile virus attacks. In *PODC 1991*, pages 51–59.

[14] T. P. Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In *CRYPTO 1991*, LNCS 576, pages 129–140.

[15] O. Reparaz, J. Balasch and I. Verbauwhede. Dude, is my code constant time? In *DATE 2017*, pages 1697–1702.

[16] A. Shamir. How to share a secret. *Commun. ACM*, 22(11):612–613, 1979.

[17] M. Tompa and H. Woll. How to share a secret with cheaters. *J. Cryptology*, 1(2):133–138, 1988.